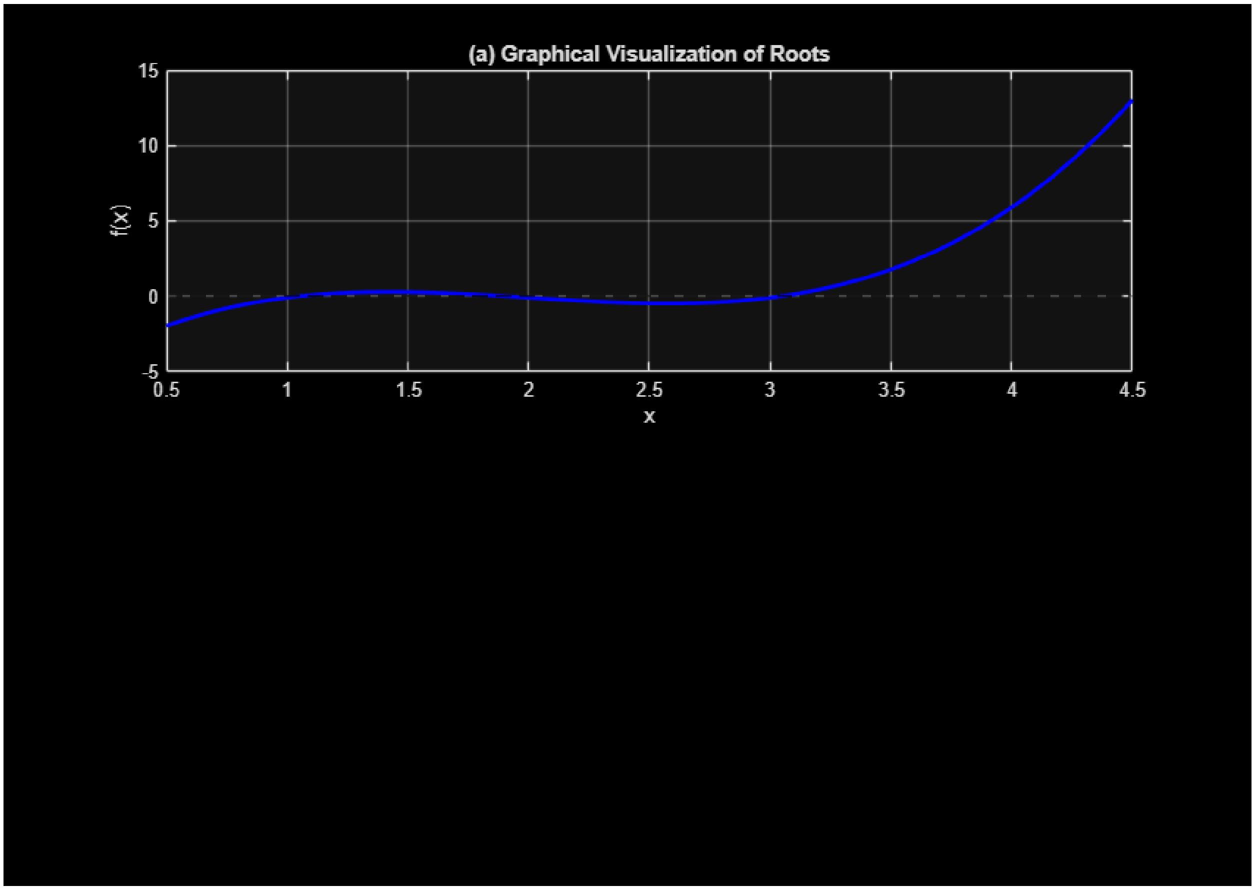
Table of Contents


```
% HW_2_3 Nouh Shaikh
clear; clc; close all;

% This was generated by AI (Google Gemini)
% This MATLAB code solves a cubic equation using seven distinct
% numerical and analytical root-finding methods. It displays step-by-step
performance
% tables and generates a log-scale chart to compare how fast each technique
converges to the root.

% f(x) = x^3 - 6*x^2 + 11*x - 6.1
f = @(x) x.^3 - 6*x.^2 + 11*x - 6.1;
df = @(x) 3*x.^2 - 12*x + 11; % Derivative for Newton-Raphson

a = -6; b = 11; c = -6.1;
tol = 1e-6;
max_iter = 30;
```

a)

```
figure('Position', [100, 100, 850, 600]);
subplot(2,1,1);
x_graph = linspace(0.5, 4.5, 1000);
plot(x_graph, f(x_graph), 'b-', 'LineWidth', 2);
hold on; yline(0, 'k--'); grid on;
xlabel('x'); ylabel('f(x)');
title('(a) Graphical Visualization of Roots');
```



b)

```
roots_b = Roots_Cubic_Equation(a, b, c);
fprintf('(b) Analytical Roots from Roots_Cubic_Equation:\n');
fprintf('    x1 = %.6f,  x2 = %.6f,  x3 = %.6f\n', roots_b(1), roots_b(2),
roots_b(3));
```

```
Q =

    0.3333


R =

   -0.0500

(b) Analytical Roots from Roots_Cubic_Equation:
    x1 = 1.054351,  x2 = 1.898969,  x3 = 3.046681
```

c)

```
fprintf('\n(c) Newton-Raphson Method Iteration Table:\n');
fprintf('Iter\t x_i\t\t f(x_i)\t\t df(x_i)\n');
```

```
x_c = 3.5;
f_history_c = [];
for iter = 1:max_iter
    fx = f(x_c);
    dfx = df(x_c);
    f_history_c(end+1) = abs(fx);
    fprintf('%2d\t %.6f\t %.6f\t %.6f\n', iter-1, x_c, fx, dfx);
    if abs(fx) < tol, break; end
    x_c = x_c - fx/dfx;
end
```

```
(c) Newton-Raphson Method Iteration Table:
Iter     x_i          f(x_i)          df(x_i)
 0      3.500000      1.775000      5.750000
 1      3.191304      0.399402      3.257618
 2      3.068699      0.051880      2.426352
 3      3.047317      0.001456      2.290617
 4      3.046681      0.000001      2.286624
 5      3.046681      0.000000      2.286620
```

d)

```
fprintf('\n(d) Secant Method Iteration Table:\n');
fprintf('Iter\t x_i\t\t f(x_i)\n');
x1 = 2.5; x2 = 3.5;
f_history_d = [abs(f(x1))];
fprintf('%2d\t %.6f\t %.6f\n', 0, x1, f(x1));
for iter = 1:max_iter
    fx1 = f(x1); fx2 = f(x2);
    f_history_d(end+1) = abs(fx2);
    fprintf('%2d\t %.6f\t %.6f\n', iter, x2, fx2);
    if abs(fx2) < tol, break; end
    x_new = x2 - fx2 * (x2 - x1) / (fx2 - fx1);
    x1 = x2; x2 = x4_placeholder(x_new);
end
```

```
(d) Secant Method Iteration Table:
Iter     x_i          f(x_i)
 0      2.500000      -0.475000
 1      3.500000      1.775000
 2      2.711111      -0.451517
 3      2.871091      -0.310108
 4      3.221923      0.502527
 5      3.004971      -0.089983
 6      3.037919      -0.019793
 7      3.047210      0.001213
 8      3.046674      -0.000015
 9      3.046681      -0.000000
```

e)

```
fprintf('\n(e) Modified Secant Method Iteration Table:\n');
fprintf('Iter\t x_i\t\t f(x_i)\n');
x_e = 3.5; delta = 0.001;
f_history_e = [];
for iter = 1:max_iter
    fx = f(x_e);
    f_perturbed = f(x_e + delta * x_e);
    f_history_e(end+1) = abs(fx);
    fprintf('%2d\t %.6f\t %.6f\n', iter-1, x_e, fx);
    if abs(fx) < tol, break; end
    x_e = x_e - (delta * x_e * fx) / (f_perturbed - fx);
end
```

```
(e) Modified Secant Method Iteration Table:
Iter     x_i          f(x_i)
 0      3.500000      1.775000
 1      3.192148      0.402154
 2      3.069356      0.053477
 3      3.047443      0.001746
 4      3.046685      0.000009
 5      3.046681      0.000000
```

f)

```
% Choosing 3 initial guesses flanking/near 3.5: x0=2.5, x1=3.0, x2=3.5
fprintf('\n(f) Inverse Quadratic Interpolation (IQI) Iteration Table:\n');
fprintf('Iter\t x_i\t\t f(x_i)\n');
x0_f = 2.5; x1_f = 3.0; x2_f = 3.5;
f_history_f = [abs(f(x0_f)), abs(f(x1_f))];
fprintf('%2d\t %.6f\t %.6f\n', 0, x0_f, f(x0_f));
fprintf('%2d\t %.6f\t %.6f\n', 1, x1_f, f(x1_f));

for iter = 2:max_iter
    y0 = f(x0_f); y1 = f(x1_f); y2 = f(x2_f);
    f_history_f(end+1) = abs(y2);
    fprintf('%2d\t %.6f\t %.6f\n', iter, x2_f, y2);
    if abs(y2) < tol, break; end

    % IQI Formula
    x_new = (y1*y2)/((y0-y1)*(y0-y2))*x0_f + ...
            (y0*y2)/((y1-y0)*(y1-y2))*x1_f + ...
            (y0*y1)/((y2-y0)*(y2-y1))*x2_f;

    x0_f = x1_f; x1_f = x2_f; x2_f = x_new;
end
```

```
(f) Inverse Quadratic Interpolation (IQI) Iteration Table:
Iter     x_i          f(x_i)
 0      2.500000     -0.475000
```

```
 1      3.000000      -0.100000
 2      3.500000      1.775000
 3      3.110815      0.159830
 4      3.044230      -0.005584
 5      3.046559      -0.000278
 6      3.046681      0.000000
```

g) initial guess 3.5

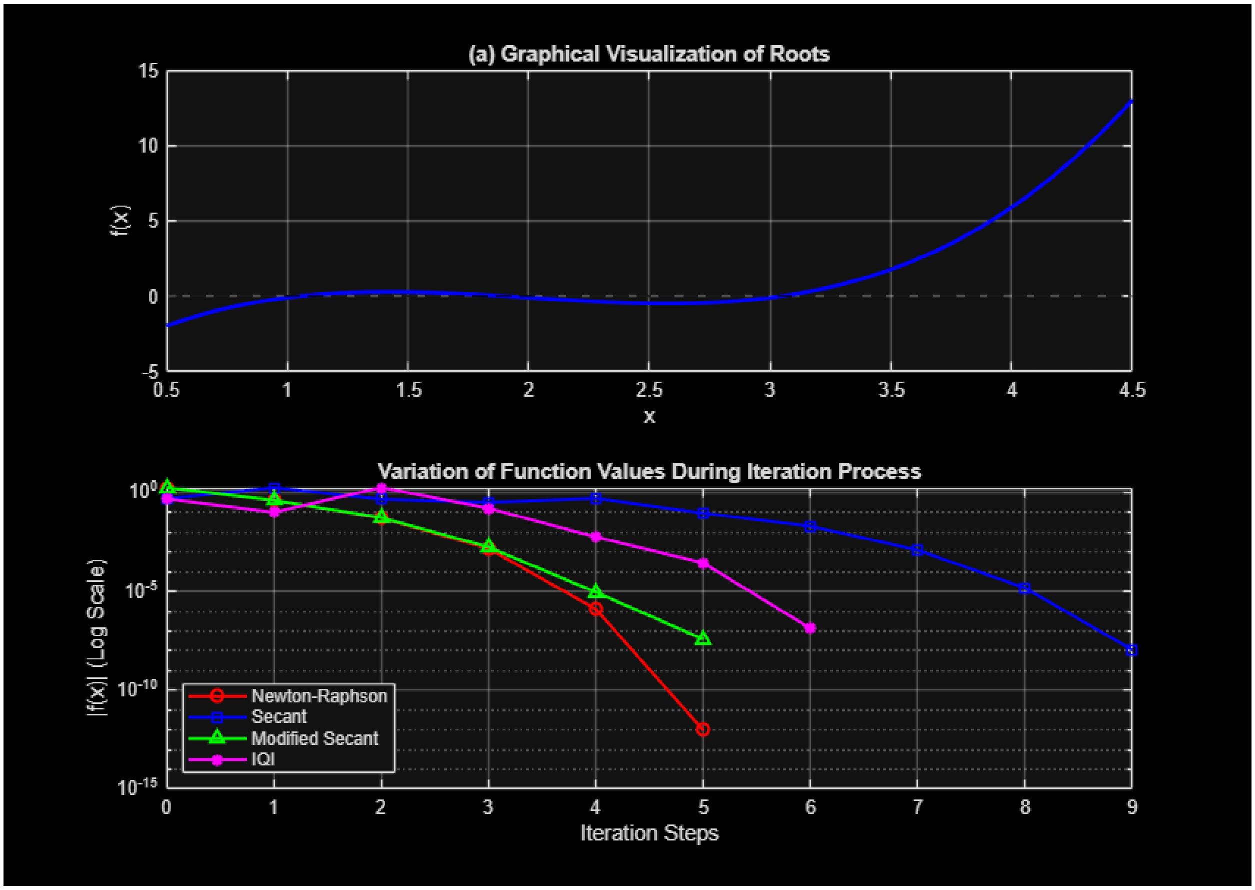
```
fprintf('\n(g) Built-In fzero Iteration Tracking:\n');
options = optimset('Display','iter','TolX',tol);
[x_g, ~, ~, output] = fzero(f, 3.5, options);
```

```
(g) Built-In fzero Iteration Tracking:

Search for an interval around 3.5 containing a sign change:
 Func-count    a          f(a)             b          f(b)        Procedure
    1             3.5         1.775           3.5         1.775    initial
interval
    3         3.40101       1.24891       3.59899       2.38929    search
    5            3.36       1.05546          3.64       2.67094    search
    7         3.30201      0.805197       3.69799        3.0976    search
    9            3.22      0.495848          3.78       3.75975    search
   11         3.10402      0.141627       3.89598       4.81957    search
   12            2.94     -0.209416       3.89598       4.81957    search

Search for a zero in the interval [2.94, 3.89598]:
 Func-count    x          f(x)             Procedure
   12            2.94     -0.209416        initial
   13         2.97981     -0.139168        interpolation
   14         3.05646     0.0226627        interpolation
   15         3.04573   -0.00218071        interpolation
   16         3.04667  -2.90523e-05        interpolation
   17         3.04668   8.91921e-10        interpolation
   18         3.04668   8.91921e-10        interpolation

Zero found in the interval [2.94, 3.89598]
```

Advanced Plotting that I don't understand lol.

```
subplot(2,1,2);
semilogy(0:length(f_history_c)-1, f_history_c, 'ro-', 'LineWidth', 1.5);
hold on;
semilogy(0:length(f_history_d)-1, f_history_d, 'bs-', 'LineWidth', 1.5);
semilogy(0:length(f_history_e)-1, f_history_e, 'g^-', 'LineWidth', 1.5);
semilogy(0:length(f_history_f)-1, f_history_f, 'm*-', 'LineWidth', 1.5);
grid on;
xlabel('Iteration Steps'); ylabel('|f(x)| (Log Scale)');
title('Variation of Function Values During Iteration Process');
legend('Newton-Raphson', 'Secant', 'Modified Secant', 'IQI', 'Location',
'SouthWest');
```

```
% Tiny helper to clean up notation
function out = x4_placeholder(in), out = in; end
```



Published with MATLAB® R2026a